

Solutions to Quiz 1: e-PGDiploma-DL&GenAI

Answer Key Summary

1-B	2-A	3-A	4-A	5-A	6-A	7-B	8-A,B,D
9-A,B,D	10-A,B,D	11-A,C,D	12-A,B,D	13-B	14-A	15-A	16-B
17-A,B,D	18-A,C,D	19-A,C,D	20-A,B,D	21-A,C,D	22-B	23-B	24-B
25-D	26-B	27-B	28-C	29-A,B,D	30-B	31-A,B,C	32-C
33-C	34-A,B,D	35-C	36-B	37-B	38-B	39-B	40-B

Detailed Solutions

Section A: Neural Network Fundamentals

Q1. Answer: B

Weighted sum:

$$z = 2(0.5) + (-1)(0.3) + 0.1 = 1 - 0.3 + 0.1 = 0.8$$

Since $\text{ReLU}(z) = \max(0, z)$,

$$\text{ReLU}(0.8) = 0.8$$

Q2. Answer: A

For sigmoid,

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Given $\sigma(z) = 0.8$,

$$\sigma'(z) = 0.8(0.2) = 0.16$$

Q3. Answer: A

Softmax probability of class 0:

$$P_0 = \frac{e^2}{e^2 + e^1 + e^0} \approx \frac{7.389}{7.389 + 2.718 + 1} \approx \frac{7.389}{11.107} \approx 0.67$$

Q4. Answer: A

Input to hidden parameters:

$$784 \times 128 + 128 = 100352 + 128 = 100480$$

Hidden to output parameters:

$$128 \times 10 + 10 = 1280 + 10 = 1290$$

Total:

$$100480 + 1290 = 101770$$

Q5. Answer: A

Binary cross-entropy for one sample with $y = 1$:

$$L = -\log(0.9) \approx 0.105$$

Q6. Answer: A

Gradient descent update:

$$w_{\text{new}} = w - \alpha \frac{\partial L}{\partial w}$$

$$w_{\text{new}} = 1.5 - 0.1(-0.4) = 1.5 + 0.04 = 1.54$$

Q7. Answer: B

Backpropagation computes gradients using the **chain rule of calculus**.

Q8. Answer: A, B, D

- A is true: ReLU has gradient 1 for positive inputs.
- B is true: ReLU is computationally cheap.
- C is false: ReLU does not always output between 0 and 1.
- D is true: ReLU can produce sparse activations.

Q9. Answer: A, B, D

- A is true: backprop computes gradients via chain rule.
- B is true: differentiability is needed (at least almost everywhere in practice).
- C is false: updates happen after forward/backward pass, not during forward pass.
- D is true: vanishing gradients can occur in deep networks.

Q10. Answer: A, B, D

- A is true: BatchNorm normalizes features approximately.
- B is true: it learns scale γ and shift β .
- C is false: it does not always remove the need for dropout.
- D is true: it often allows higher learning rates.

Q11. Answer: A, C, D

- A: dropout helps reduce overfitting.
- B: increasing model size often worsens overfitting.
- C: early stopping helps.

- D: L2 regularization helps.

Q12. Answer: A, B, D

- A: multiclass classification $\rightarrow$ cross-entropy is correct.
- B: binary classification $\rightarrow$ BCE is correct.
- C: regression $\rightarrow$ softmax loss is incorrect.
- D: multilabel classification $\rightarrow$ BCE per label is correct.

Section B: CNN Concepts and Computation

Q13. Answer: B

Output size formula:

$$\left\lfloor \frac{32 + 2(1) - 5}{2} \right\rfloor + 1 = \left\lfloor \frac{29}{2} \right\rfloor + 1 = 14 + 1 = 15$$

So output is 15×15 .

Q14. Answer: A

Trainable parameters:

$$32 \times (16 \times 3 \times 3) + 32 = 32 \times 144 + 32 = 4608 + 32 = 4640$$

Q15. Answer: A

The top-left 2×2 block is:

$$\begin{bmatrix} 5 & 3 \\ 2 & 7 \end{bmatrix}$$

Max pooling selects the maximum, so the output at $(0,0)$ is:

$$7$$

Q16. Answer: B

Two consecutive 3×3 convolutions with stride 1 give an effective receptive field:

$$5 \times 5$$

Q17. Answer: A, B, D

- A: true, due to parameter sharing.
- B: true, convolutions are translation equivariant.
- C: false, fully connected layers capture global context more directly in one layer.
- D: true, convolutions exploit local spatial structure.

Q18. Answer: A, C, D

- A: true, max pooling keeps dominant features.
- B: false, pooling has no trainable parameters.

- C: true, it reduces spatial dimensions and computation.
- D: true, average pooling computes means and is often useful near final layers.

Q19. Answer: A, C, D

- A: true, 1×1 convolution can reduce channels.
- B: false, it does not increase spatial resolution.
- C: true, with activation it adds non-linearity.
- D: true, it learns cross-channel interactions.

Q20. Answer: A, B, D

- A: freezing early layers is common.
- B: replacing the final classifier is standard.
- C: false, high learning rate on the full pretrained network is usually harmful.
- D: true, smaller LR for pretrained layers is good practice.

Q21. Answer: A, C, D

- A: common augmentation.
- B: shuffling pixels destroys image structure.
- C: common augmentation.
- D: Mixup is a valid augmentation strategy.

Q22. Answer: B

ResNet shortcut connections primarily address the **degradation problem**, where deeper plain networks can perform worse even on training data.

Q23. Answer: B

The first 1×1 convolution in a bottleneck block reduces channel dimensionality before the expensive 3×3 convolution.

Q24. Answer: B

The Inception module applies multiple kernel sizes and pooling in parallel, then concatenates the outputs channel-wise.

Q25. Answer: D

All statements A, B, and C are correct about MobileNetV2:

- inverted residual structure,
- depthwise separable convolution,
- linear bottleneck without final ReLU.

Q26. Answer: B

Using a very high learning rate on a frozen backbone is the **least effective** strategy.

Q27. Answer: B

If gradients are zero from the first epoch, the most likely issue is a **dead ReLU** problem.

Section C: Encoder-Decoder and Sequence Models

Q28. Answer: C

Q29. Answer: A, B, D

Q30. Answer: B

Q31. Answer: A, B, C

Q32. Answer: C

Q33. Answer: C

Q34. Answer: A, B, D

Q35. Answer: C

Section D: PyTorch and Training Dynamics

Q36. Answer: B

After convolution and pooling, the tensor shape is:

$$[N, 16, 14, 14]$$

This should be flattened as:

$$x = x.view(x.size(0), -1)$$

Using `x.view(-1)` collapses the batch dimension and causes a mismatch for `Linear`.

Q37. Answer: B

The bug is:

```
optimizer.step()
```

is called before

```
loss.backward()
```

So the optimizer updates parameters before fresh gradients are computed.

Q38. Answer: B

Input shape: `[8, 3, 64, 64]`

After first conv:

$$[8, 16, 64, 64]$$

After first pool:

$$[8, 16, 32, 32]$$

After second conv:

$$[8, 32, 32, 32]$$

After second pool:

[8, 32, 16, 16]

Q39. Answer: B

Initial learning rate:

0.001

With `StepLR(step_size=2, gamma=0.1)`, after epoch 2 it becomes:

$$0.001 \times 0.1 = 0.0001$$

After epoch 3 it remains:

0.0001

Q40. Answer: B

`model.eval()` switches layers like Dropout and BatchNorm to inference mode, but gradient tracking is still active unless `torch.no_grad()` is used.